

Great choice. Setting up a bulletproof baseline is the most critical step of an independent research project—if your benchmark isn't robust, your machine learning improvements won't hold up under peer review.

For our traditional econometrics baseline, we will focus on implementing two distinct flavors of forecasting: **Univariate Parametric Models (GARCH/EGARCH)** using daily returns, and **Non-Parametric High-Frequency Models (HAR-RV)** if you have access to intraday data.

Here is the step-by-step roadmap to build and implement Step 1 in Python.

1. Data Preparation & Returns Calculation

Before running any volatility model, you need to clean your equity data and calculate log returns. Log returns are preferred in econometrics because they are time-additive.

Python

```
import numpy as np
```

```
import pandas as pd
```

```
import yfinance as yf # Using yfinance as a quick source for equity data
```

```
# 1. Download historical equity data (e.g., Apple)
```

```
ticker = "AAPL"
```

```
data = yf.download(ticker, start="2016-01-01", end="2026-01-01")
```

```
# 2. Calculate Log Returns
```

```
data['Log_Return'] = np.log(data['Close'] / data['Close'].shift(1))
```

```
data.dropna(inplace=True)
```

```
# Scale returns by 100 to help GARCH optimization algorithms converge smoothly
```

```
data['Scaled_Return'] = data['Log_Return'] * 100
```

2. Model A: GARCH(1,1) & EGARCH(1,1)

The arch library in Python is the standard package for modeling autoregressive conditional heteroskedasticity.

The Math Refresher

- **Standard GARCH(1,1):** Symmetric tracking of variance.
- **EGARCH(1,1) (Exponential GARCH):** Models the *log* of variance, meaning parameters don't have to be strictly positive, and it includes an asymmetry parameter (γ) to capture the **leverage effect** (where market drops cause higher volatility spikes than market gains).

Python Implementation

Python

```
from arch import arch_model
```

```
# 1. Fit a Standard GARCH(1,1) model
```

```
# vol='Garch' specifies the volatility structure; p=1, q=1 sets the lags
```

```
garch_spec = arch_model(data['Scaled_Return'], vol='Garch', p=1, q=1, dist='Normal')
```

```
garch_result = garch_spec.fit(dispatch='off')
```

```
print(garch_result.summary())
```

```
# 2. Fit an EGARCH(1,1) model to capture the leverage effect
```

```
egarch_spec = arch_model(data['Scaled_Return'], vol='EGARCH', p=1, q=1, dist='Normal')
```

```
egarch_result = egarch_spec.fit(dispatch='off')
```

```
# 3. Extract conditional volatility (annualized)
```

```
data['GARCH_Vol'] = garch_result.conditional_volatility
```

```
data['EGARCH_Vol'] = egarch_result.conditional_volatility
```

3. Model B: The HAR-RV Model (If using Intraday Data)

If your research project leverages high-frequency tick data, the **Heterogeneous Autoregressive model of Realized Volatility (HAR-RV)** is a mandatory benchmark. It uses OLS regression to forecast tomorrow's volatility based on past daily, weekly (5-day average), and monthly (22-day average) realized volatilities.

Python Implementation

If you already have a column of daily **Realized Volatility (\$RV\$)** calculated from 5-minute intraday intervals, you structure the HAR model like this:

Python

```
import statsmodels.api as sm
```

```
# Assuming 'RV' is already calculated and present in your dataframe:
```

```
# 1. Create rolling weekly and monthly averages
```

```
data['RV_Daily'] = data['RV']
```

```
data['RV_Weekly'] = data['RV'].rolling(window=5).mean()
```

```
data['RV_Monthly'] = data['RV'].rolling(window=22).mean()
```

```
# Tomorrow's RV is our target variable
```

```
data['Target_RV'] = data['RV'].shift(-1)
```

```
data.dropna(inplace=True)
```

```
# 2. Define features and target
```

```
X = data[['RV_Daily', 'RV_Weekly', 'RV_Monthly']]
```

```
X = sm.add_constant(X) # Add intercept ( $\omega$ )
```

```
y = data['Target_RV']
```

```
# 3. Fit via Ordinary Least Squares (OLS)
```

```
har_model = sm.OLS(y, X).fit()
```

```
print(har_model.summary())
```

```
data['HAR_Forecast'] = har_model.predict(X)
```

4. Setting Up the Evaluation Metrics

To eventually see if your Machine Learning models beat these baselines, we need to establish our evaluation metrics. For volatility, **QLIKE** is heavily favored over standard Mean Squared Error (MSE) because it penalizes underprediction more severely than overprediction, which is essential for risk management.

Python

```
def calculate_qlike(actual_variance, predicted_variance):  
    """  
    Computes the QLIKE loss function.  
    Both inputs should be in variance terms (volatility squared).  
    """  
    return np.mean(actual_variance / predicted_variance - np.log(actual_variance /  
predicted_variance) - 1)  
  
# Example comparison (using squared conditional volatilities as variance proxies)  
# actual_proxy = data['Scaled_Return'] ** 2 # Daily squared returns as a noisy proxy  
# garch_variance = data['GARCH_Vol'] ** 2  
# qlike_garch = calculate_qlike(actual_proxy, garch_variance)
```

Moving Forward

Do you want to focus our dataset on **daily end-of-day data** (using GARCH/EGARCH) or do you have access to **high-frequency intraday data** to compute Realized Volatility for the HAR model?

Once we settle on the exact data structure, we can write a clean rolling walk-forward cross-validation loop to generate true *out-of-sample* baseline forecasts.